

Documentazione di Progettazione Object-Oriented

Sistema Informativo per la Gestione di Campagne di Giochi di Ruolo (GDR)

Corso di Programmazione Object Oriented e Basi di Dati — A.A. 2025-2026

Svolto da: Pontillo Salvatore, Riccardi Carmine

Indice degli argomenti

Questo documento descrive la progettazione object-oriented del sistema realizzato per il progetto d'esame: l'architettura usata, le scelte fatte riguardo ai principi OOP, la gestione delle eccezioni e il funzionamento dei casi d'uso principali tramite i diagrammi di sequenza. Il documento e' organizzato cosi':

- Sezione 1 — Introduzione e analisi del dominio applicativo
- Sezione 2 — Architettura del sistema (pattern BCE e DAO)
- Sezione 3 — Principi Object-Oriented e scelte progettuali
- Sezione 4 — Dizionario delle classi principali
- Sezione 5 — Gestione delle eccezioni
- Sezione 6 — Diagrammi di sequenza dei casi d'uso principali
- Sezione 7 — Diagramma delle classi UML del dominio

1. Introduzione e Analisi del Dominio

Il progetto consiste nella realizzazione di un sistema informativo per la gestione centralizzata di campagne di Giochi di Ruolo (GDR). Il dominio e' stato pensato per due tipi di utenti:

- **Master:** e' il creatore e l'arbitro della campagna. Puo' creare Razze, Classi e Oggetti, gestire i partecipanti e modificare le statistiche o l'inventario di un personaggio quando serve per la storia.
- **Giocatore:** e' chi partecipa alla campagna. Puo' iscriversi alle campagne create dai Master, creare il proprio Personaggio Giocante (PG) rispettando le regole di classe e razza, gestire le proprie risorse (HP, Mana, Oro) e comprare oggetti nel negozio virtuale.

2. Architettura del Sistema: BCE e DAO

Il software e' stato organizzato seguendo un'architettura a livelli, unendo il pattern BCE (Boundary-Control-Entity) con il pattern DAO (Data Access Object) per la persistenza dei dati. In questo modo i package sono piu' coesi al loro interno e meno legati tra loro, rendendo il sistema piu' facile da mantenere e modificare.

2.1. Pattern BCE

- **Boundary (package gui):** e' l'interfaccia grafica, realizzata con Java Swing. Si occupa solo di mostrare i dati e raccogliere gli eventi dell'utente, senza contenere logica di business o query SQL: si limita a chiamare il livello Control.
- **Control (package controller):** la classe Controller gestisce la logica dell'applicazione. Tiene lo stato della sessione (utente loggato, campagna attiva), controlla le richieste che arrivano dalla GUI e coordina le operazioni tra le Entity e il database.
- **Entity (package model):** rappresentano le regole e i dati del gioco. Non dipendono ne' dalla grafica ne' dal database.

2.2. Pattern DAO

Per la persistenza su PostgreSQL e' stato usato il pattern DAO, organizzato in tre package:

- **dao:** contiene le interfacce (es. UtenteDAO, PersonaggioDAO, InventarioDAO) con le operazioni CRUD, senza dipendere da un DBMS specifico.
- **dao.impl:** contiene le classi che implementano queste interfacce per PostgreSQL (es. ImplementazionePostgresUtente, ImplementazionePostgresGiocatore, ImplementazionePostgresInventario).
- **db:** gestisce la connessione al database, cioe' apertura e chiusura della Connection JDBC.

Grazie a questa divisione, per cambiare tecnologia di persistenza basterebbe scrivere nuove classi nel package dao.impl, senza dover toccare Controller ed Entity.

3. Principi Object-Oriented e Scelte Progettuali

Il dominio e' stato modellato usando i principi della programmazione a oggetti, puntando soprattutto sul riuso del codice e sulla protezione dei dati.

3.1. Ereditarieta' e Polimorfismo

- **Gerarchia Utente:** e' stata creata una classe astratta Utente con gli attributi comuni (username, password, email), da cui derivano le classi Giocatore e Master. Questo permette al Controller di usare un riferimento generico (`private Utente utenteAttivo;`) e di usare l'operatore `instanceof` per capire quale GUI mostrare.
- **Gerarchia Oggetto:** esiste una classe astratta Oggetto (id, nome, costo), da cui derivano `OggettoConsumabile`, con i parametri di cura, e `OggettoEquipaggiabile`, con requisiti e bonus statistici. Il polimorfismo permette di gestire le transazioni del negozio in modo uniforme.

3.2. Incapsulamento e Protezione dello Stato

Gli attributi delle classi Entity sono privati e si accede a loro solo tramite getter e setter. Ad esempio, nella classe Personaggio l'oro e le risorse vitali non possono mai scendere sotto zero, e i metodi per curare il personaggio non fanno mai superare i valori massimi previsti dalle statistiche.

3.3. Riuso, Estendibilita' e Separazione delle Responsabilita'

- **Riuso del codice:** la classe Statistica e' l'esempio principale. Invece di ripetere gli stessi attributi numerici (forza, destrezza, ecc.) in Personaggio, Razza e OggettoEquipaggiabile, questi sono raccolti in un'unica classe usata in contesti diversi, riducendo le ripetizioni.
- **Estendibilita':** le gerarchie Utente e Oggetto sono pensate per poter aggiungere in futuro nuove sottoclassi (es. un ruolo Osservatore o un nuovo tipo di oggetto) senza dover modificare il Controller, che lavora gia' sui tipi astratti.
- **Separazione delle responsabilita':** ogni livello dell'architettura (GUI, Controller, Entity, DAO) cambia solo per un motivo preciso, ad esempio la GUI cambia solo per motivi grafici e i DAO solo se cambia la persistenza. Questo rende piu' semplice sostituire un livello senza toccare gli altri.

4. Dizionario delle Classi Principali

Entity	Descrizione e responsabilita'
Utente, Giocatore, Master	La classe astratta Utente contiene le credenziali di accesso (username, password, email). Da essa derivano Master, che ha i permessi per creare il mondo di gioco, e Giocatore, che tiene lo storico delle iscrizioni e i Personaggi Giocanti (PG) creati.
Campagna	E' l'entita' che raggruppa tutto il resto. Gestisce lo stato (Iniziata/Non Iniziata) per permettere o bloccare le iscrizioni, e contiene le liste di Partecipanti, Personaggi, Classi, Razze e Oggetti. Ogni Campagna ha un solo Master.
Personaggio	E' il cuore del gioco. Contiene il metodo <code>ricalcolaStatisticheFinali()</code> , che somma le statistiche base ai bonus dell'equipaggiamento indossato e toglie automaticamente gli oggetti se i requisiti non sono piu' rispettati. Gestisce anche l'inventario.
Classe e Razza	Definiscono le caratteristiche del personaggio. La Razza da' dei modificatori fissi alla creazione. La Classe fornisce le abilita' sbloccabili e l'equipaggiamento iniziale.

Entity	Descrizione e responsabilita'
Statistica	E' la classe che mette in pratica il riuso del codice: invece di ripetere gli stessi attributi numerici (forza, destrezza, ecc.) in piu' classi, sono raggruppati qui e riutilizzati come profilo del Personaggio, modificatore della Razza e requisiti/bonus dell'OggettoEquipaggiabile.
Oggetto, OggettoConsumabile, OggettoEquipaggiabile	Gerarchia astratta (id, nome, costo) da cui derivano OggettoConsumabile, con i parametri di cura, e OggettoEquipaggiabile, con requisiti e bonus statistici. Grazie al polimorfismo le transazioni del negozio vengono gestite in modo uniforme.

5. Gestione delle Eccezioni

E' stato creato il package exception, che contiene una gerarchia di eccezioni checked personalizzate (es. AutenticazioneException, OggettoNonSelezionatoException, PersonaggioNonTrovatoException, NomePgNonValidoException).

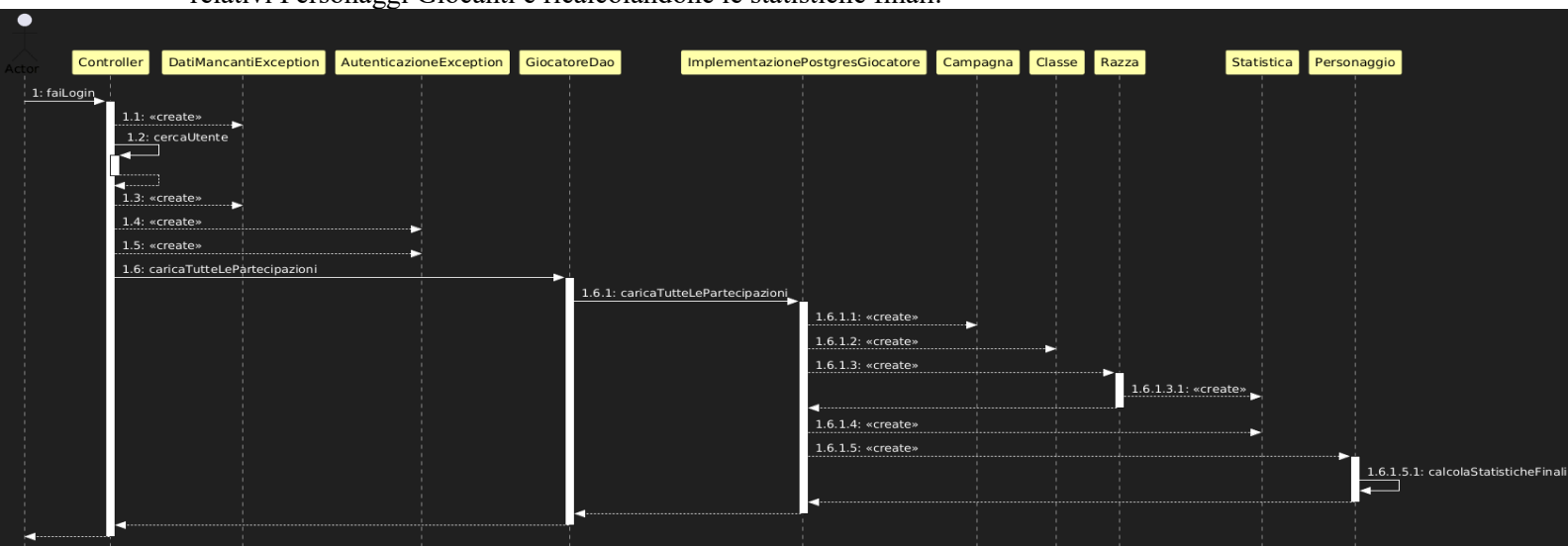
- **Gestione degli errori:** se una query viola i vincoli del database, il DAO intercetta la SQLException e lancia un'eccezione a runtime. Il Controller la gestisce e la trasforma in un messaggio comprensibile per la GUI, senza mai mostrare all'utente errori tecnici come SQLException o NullPointerException.
- **Transazioni:** nelle operazioni piu' delicate, come l'acquisto di un oggetto, l'autocommit viene disabilitato. Se qualcosa va storto durante l'operazione, viene eseguito un rollback() per lasciare il database in uno stato coerente.

6. Diagrammi di Sequenza dei Casi d'Uso

Per mostrare come funzionano le operazioni piu' importanti attraverso i vari livelli (BCE + DAO), sono stati realizzati i seguenti diagrammi di sequenza.

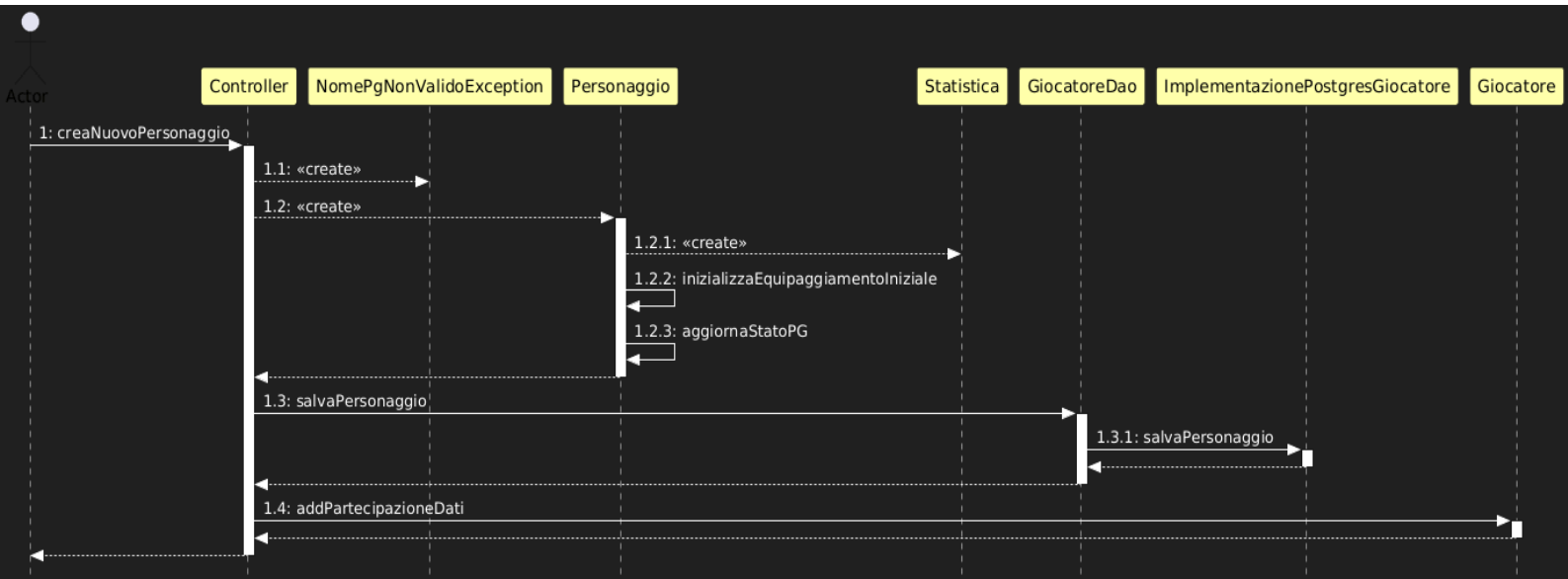
6.1. Autenticazione (faiLogin)

Il diagramma mostra il processo di login. Il Controller verifica le credenziali; se l'utente e' un Giocatore, il sistema interroga il GiocatoreDao per recuperare le campagne a cui e' iscritto, caricando anche i relativi Personaggi Giocanti e ricalcolandone le statistiche finali.



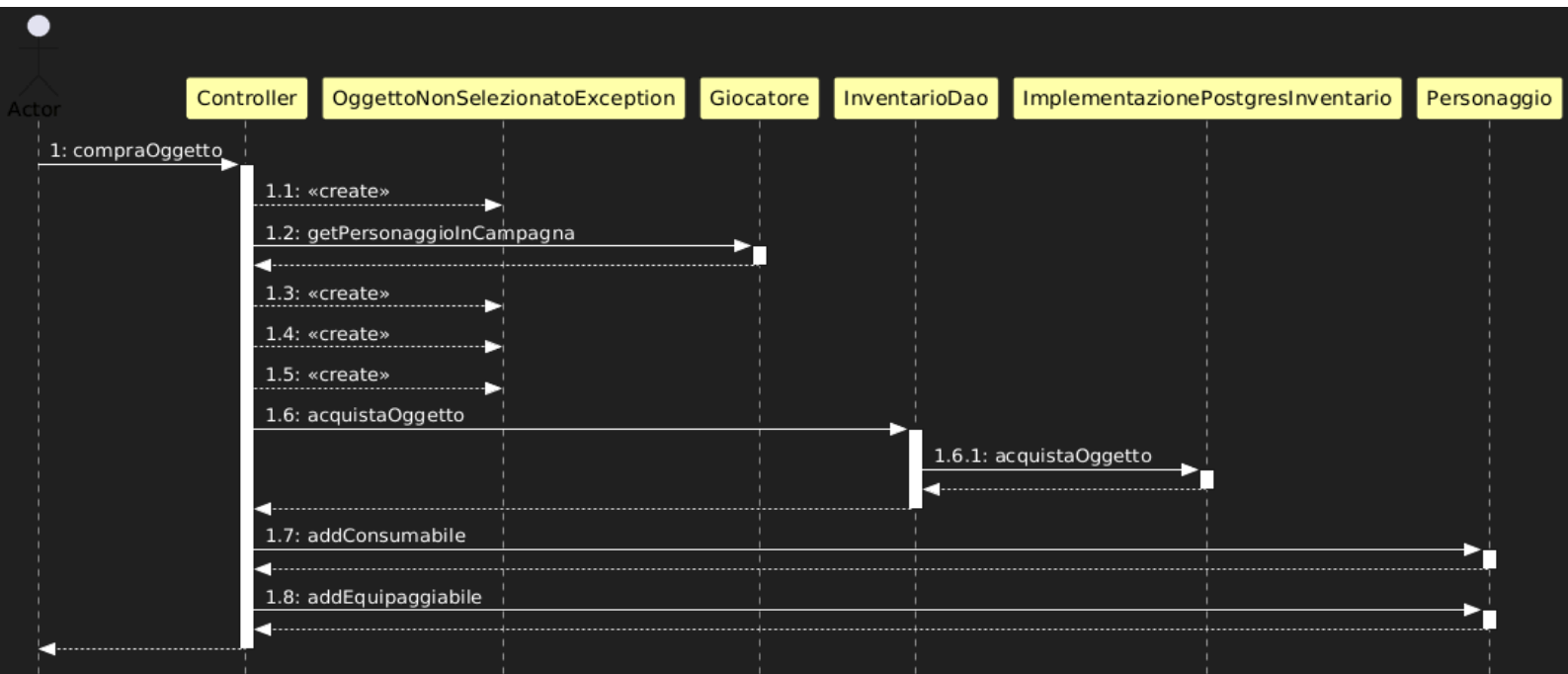
6.2. Creazione Personaggio (creaNuovoPersonaggio)

Mostra cosa succede quando un giocatore crea il primo personaggio in una campagna. Il Controller crea un nuovo Personaggio con l'equipaggiamento di base, lo salva tramite il GiocatoreDao e associa l'istanza creata allo stato del giocatore in memoria.



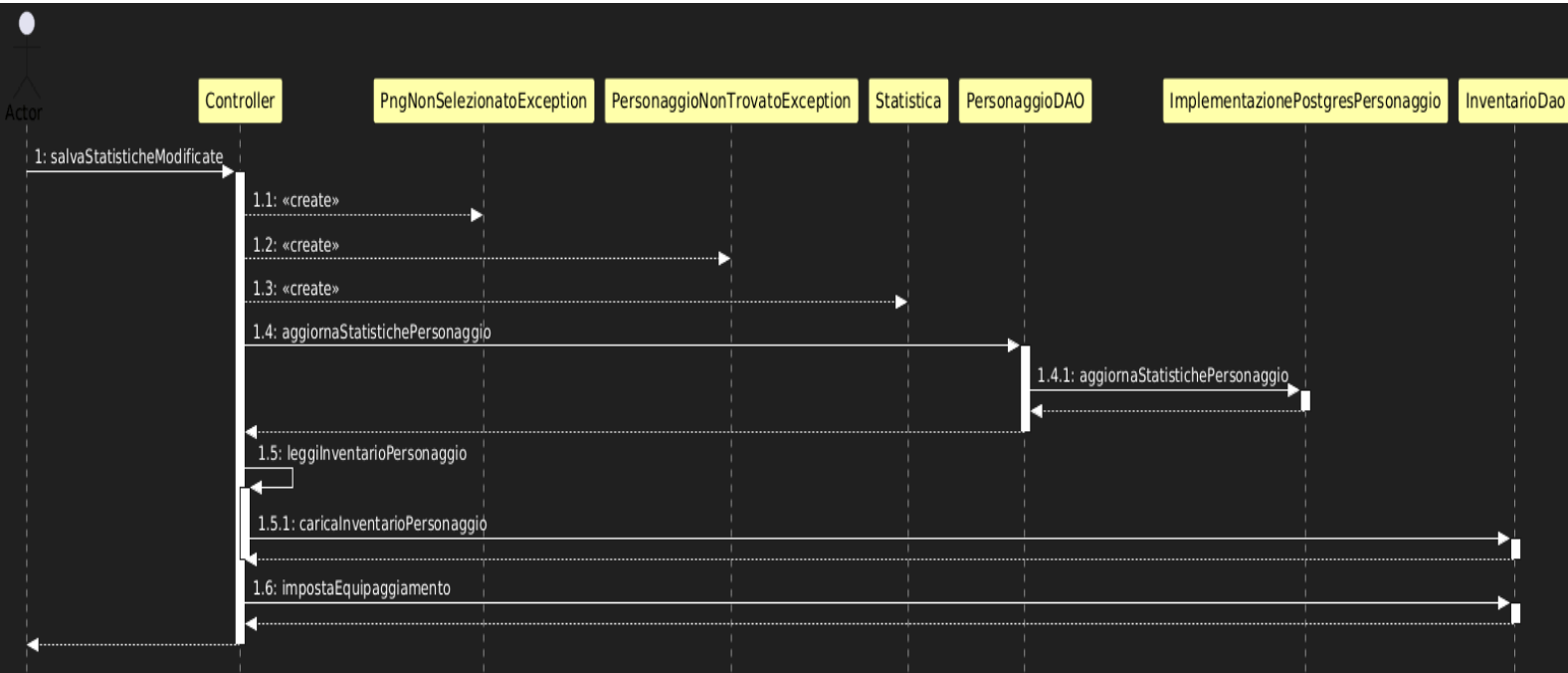
6.3. Acquisto Oggetto (compraOggetto)

Mostra la transazione economica nel negozio. Il Controller controlla che il giocatore abbia abbastanza oro e che non possieda già l'oggetto; se i controlli vanno a buon fine, viene chiamato l'InventarioDao che esegue l'operazione sul database e solo dopo l'oggetto viene aggiunto all'inventario in memoria.



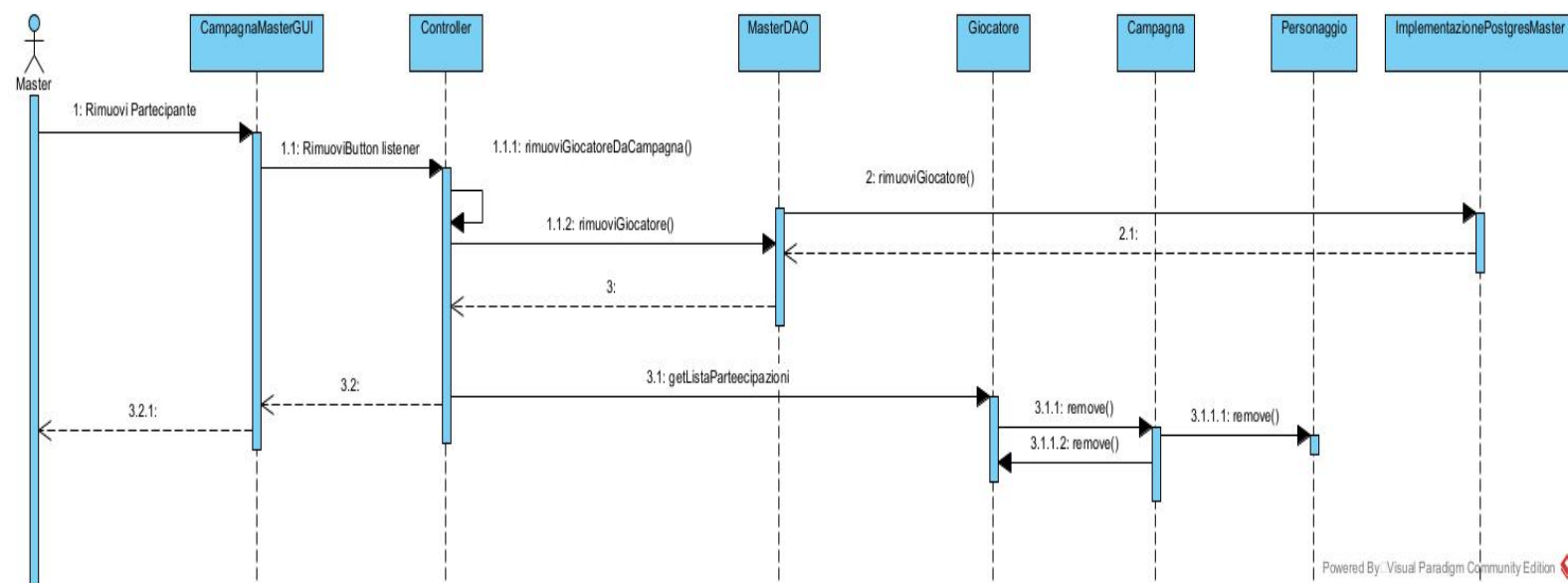
6.4. Override Statistiche Master (salvaStatisticheModificate)

Mostra un'operazione riservata al Master. Quando le statistiche di un PG vengono forzate manualmente, il Controller richiede l'aggiornamento al PersonaggioDAO e poi ricontrolla l'inventario: se le nuove statistiche non soddisfano piu' i requisiti dell'equipaggiamento indossato, l'oggetto viene tolto automaticamente tramite l'InventarioDao.



6.5. Rimuovi Partecipante (rimuoviGiocatoreDaCampagna)

Mostra un'operazione amministrativa e di gestione esclusiva del Master. Tramite la CampagnaMasterGUI, il Master seleziona un utente e richiede la rimozione del partecipante: il Controller coordina l'operazione delegando al MasterDAO (e alla sua implementazione concreta PostgreSQL) l'eliminazione della partecipazione sul database; successivamente, il sistema aggiorna lo stato in memoria rimuovendo le relative associazioni dalle collezioni della Campagna e i conseguenti riferimenti ai Personaggio collegati.



Il diagramma seguente mostra le relazioni tra le classi del dominio (associazioni, aggregazioni, composizioni e generalizzazioni).

